Performance Testing

Date	30 September 2026
Team ID	6ab3ec14b5170a9b58b3a658
Project Name	Implement Client Script and UI Policy
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter (HTTP load generation against the ServiceNow instance); browser Developer Tools (Network tab) to time form and list page loads
Type of Testing	Load Testing (normal and peak load) and Spike Testing
Target Module / API	ServiceNow Incident table: New Incident form (incident.do), Incident list view (incident_list.do) and Table API endpoint /api/now/table/incident, with the High Impact Control UI Policy and the onChange, onSubmit and onCellEdit Client Scripts active
Test Environment	Cloud: hosted ServiceNow instance, accessed from a local laptop over the internet
Test Date	ServiceNow instance

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline: open a New Incident form (incident.do), set Impact to High and save with Assignment Group and Assigned To filled in	1	60	Form loads in under 2 seconds; Assignment Group becomes mandatory; Urgency is auto-set to High and read-only; the record saves.
2	Normal load: users create High Impact Incidents with valid data (Impact, Urgency, Assignment Group, Assigned To) through the Table API	10	120	All requests succeed; average response time under 2 seconds; error rate under 1%.
3	Peak load: mixed traffic of Incident creation and Incident list view loads (incident_list.do) with the onCellEdit script active	25	180	Maximum response time under 5 seconds; error rate under 1%; CPU and memory under 80%; list-view State edit is still blocked with an alert.
4	Spike: sudden burst of Incident creation requests after a quiet period	50	60	Instance stays available and response times return to normal once the spike ends; error rate under 1%.

Step 4: Observations & Analysis

Key Findings:

- The controls under test (High Impact Control UI Policy, onChange, onSubmit and onCellEdit Client Scripts) run in the browser and use only simple g_form calls (getValue, setValue, addInfoMessage, showErrorBox). None of them makes a server call, so the server-side cost under load is the standard Incident insert.
- UI Policies and Client Scripts are not executed for REST API requests, so the Table API scenarios measure the server-side Incident save only. The client-side rules were verified functionally in the Chapter 7 test activities.

Bottlenecks Identified:

Optimization Steps Taken:

- Rules for mandatory and read-only fields are handled by a declarative UI Policy (with Reverse if false enabled) instead of custom script code.
- The onChange script exits early when the form is loading or the new value is empty (isLoading check), so it does not run needlessly when the form opens.

Step 5: Screenshots / Evidence



